

How-to Guides

Prompt engineering

What is prompt engineering?

Prompt engineering is a technique used in natural language processing (NLP) to improve the performance of large language models (LLMs) by providing them with more context and information about the task in hand. It involves creating prompts—short pieces of text—to provide additional information or guidance to the model to produce more accurate and relevant results.

Prompt engineering takes your Llama applications beyond basic interactions to create sophisticated, production-ready systems. While you can improve model performance through [fine-tuning](#), [distillation](#), or [upgrading to larger or newer models](#), optimizing your prompts often provides the fastest path to better results—achieving the performance improvements you need without additional model training or infrastructure costs.

Crafting effective prompts

Creating effective prompts is an important part of prompt engineering. Here are some tips for creating prompts that will help improve the performance of your language model:

1. **Be clear and concise:** Your prompt should be easy to understand and provide enough information for the model to generate relevant output. Avoid using jargon or technical terms that may confuse the model.
2. **Use specific examples:** Providing specific examples ("few-shot") in your prompt can help the model better understand what kind of output is expected. For example, if you want the model to generate a story about a particular topic, include a few sentences about the setting, characters, and plot.
3. **Vary the prompts:** Using different prompts can help the model improve at its task and produce more diverse and creative output. Try using different styles, tones, and formats to see how the model responds.
4. **Test and refine:** Once you have created a set of prompts, test them out on the model to see how it performs. If the results are not as expected, try refining the prompts by adding

more detail or adjusting the tone and style.

5. **Gather feedback:** Finally, use feedback from users or other sources to continually improve your prompts. This can help you identify areas where the model needs more guidance and make adjustments accordingly.

Detailed, explicit instructions produce better results than open-ended prompts. Giving explicit instructions is like placing rules and restrictions on how the model responds to your prompt.

Stylization

You can steer the model towards responding in a specific writing style:

Explain this to me like a topic on a children's educational network show teaching elementary students.



I'm a software engineer using large language models for summarization. Summarize the following text in under 250 words:



Give your answer like an old-timey private investigator hunting down a case step-by-step.



Formatting

You can request specific formats using prompts:

Use bullet points.



Return as a JSON object.



Use fewer technical terms and help me apply it in my work in communications.



Restrictions

Restrictions tell the model what not to do:

Only use academic papers.



Never give sources older than 2020.



If you don't know the answer, say that you don't know.



The example below illustrates how explicit instructions give more specific results by limiting the responses to recently created sources.

More likely to cite sources from 2017:

Explain the latest advances in large language models to me.



Gives more specific advances and only cites sources from 2020:

Explain the latest advances in large language models to me.



Always cite your sources. Never cite sources older than 2020.

Prompting techniques

Zero- and few-shot prompting

A shot is an example or demonstration of what type of prompt and response you expect from a large language model. This term originates from training computer vision models on photographs, where one shot was one example or instance that the model used to classify an image.

Zero-shot prompting

Modern LLMs like Llama are capable of following instructions and producing responses without having previously seen an example of a task. Prompting without examples is called "zero-shot prompting".

In the following example, the model is able to analyse the sentiment of simple text without needing examples of previous analyses:

Text: This was the best movie I've ever seen!
The sentiment of the text is:



Text: The director was trying too hard.
The sentiment of the text is:



Few-shot prompting

Adding specific examples of your desired output generally results in a more accurate, consistent output when compared with zero-shot prompting. This technique is called "few-shot prompting".

In this example, the generated response follows our desired format that offers a more nuanced sentiment classifier that gives a positive, neutral, and negative response confidence percentage.

You are a sentiment classifier. For each message, give the percentage of positive/neutral/negative.



Here are some samples:

Text: I liked it

Sentiment: 70% positive 30% neutral 0% negative

Text: It could be better

Sentiment: 0% positive 50% neutral 50% negative

Text: It's fine

Sentiment: 25% positive 50% neutral 25% negative

Text: I thought it was okay

Text: I loved it!

Text: Terrible service 0/10

Role-based prompts

Creating prompts based on the role or perspective of the person or entity being addressed can be useful for generating more relevant and engaging responses from the model.

Pros:

- **Improves relevance:** Role-based prompting helps the language model understand the role or perspective of the person or entity being addressed, which can lead to more relevant and engaging responses.

- **Increases accuracy:** Providing additional context about the role or perspective of the person or entity being addressed can help the language model avoid making mistakes or misunderstandings.

Cons:

- **Requires effort:** Requires more effort to gather and provide the necessary information about the role or perspective of the person or entity being addressed.

Example

You are a virtual tour guide walking tourists around Eiffel Tower on a night tour. Describe Eiffel Tower to your audience in a way that includes its history, the number of people visiting each year, the amount of time it takes to do a full tour and why so many people visit it each year.

Chain-of-thought prompting

Chain-of-thought prompting provides the language model with a series of prompts or questions to help guide its thinking and generate a more coherent and relevant response. This technique can be useful for generating more thoughtful and well-reasoned responses from language models.

Pros:

- **Improves coherence:** Helps the language model think through a problem or question in a logical and structured way, which can lead to more coherent and relevant responses.
- **Increases depth:** Providing a series of prompts or questions can help the language model explore a topic more deeply and thoroughly, potentially leading to more insightful and informative responses.

Cons:

- **Requires effort:** The chain of thought technique requires more effort to create and provide the necessary prompts or questions.

Example

You are a virtual tour guide from 1901. You are guiding tourists visiting Eiffel Tower. Describe Eiffel Tower to your audience.

Begin with:

1. Why it was built
2. Then by how long it took them to build
3. Where were the materials sourced to build

4. Number of people it took to build

End with the number of people visiting the Eiffel tower annually in the 1900's, the amount of time it takes to complete a full tour and why so many people visit this place each year.

Make your tour funny by including 1 or 2 funny jokes at the end of the tour.

Self-consistency

LLMs are probabilistic; even with chain-of-thought prompting, a single generation might produce incorrect results. Self-consistency introduces enhanced accuracy by selecting the most frequent answer from multiple generations, at the cost of higher compute.

Example

John found that the average of 15 numbers is 40.
If 10 is added to each number then the mean of the numbers is?
Report the answer surrounded by three backticks, for example:
```123```

Running the above several times and taking the most commonly returned value for the answer would make use of the self-consistency approach.

## Retrieval-augmented generation

Modern models are generally able to produce common facts out-of-the-box, using just the model weights.

What is the capital of California?  
# The capital of California is Sacramento...

But unless a model was trained or [fine-tuned](#) on domain-specific or time-specific data, it is unlikely to produce accurate facts outside of its training knowledge.

What was the temperature in Menlo Park on December 12th, 2023?  
# I'm just an AI, I don't have access to real-time or historical weather data...

Retrieval-augmented generation (RAG) describes the practice of including information in the prompt that has been retrieved from an external database. It's an effective way to incorporate facts into your LLM application and is more affordable than [fine-tuning](#), which might also negatively impact the foundational model's capabilities.

The information source in a RAG system could be as simple as a lookup table or as sophisticated as a vector database containing all of your company's knowledge:

Given the following information about temperatures in Menlo Park:

2023-12-11 : 52 degrees Fahrenheit

2023-12-12 : 51 degrees Fahrenheit

2023-12-13 : 55 degrees Fahrenheit

What was the temperature in Menlo Park on 2023-12-12?

# Sure! The temperature in Menlo Park on 2023-12-12 was 51 degrees Fahrenheit.

What was the temperature in Menlo Park on 2023-07-18 ?

# Sorry, I don't have information about the temperature in Menlo Park on 2023-07-18. The information provided only includes temperatures for December 11th, 12th, and 13th of 2023.

## Limiting extraneous tokens

A common challenge in LLM applications is ensuring a model generates a suitable response without extraneous tokens (e.g. "Sure! Here's more information onâ€¦").

By combining a role, rules and restrictions, explicit instructions, and an example, the model can be prompted to generate the desired response.

## Example

Prompt:

You are a robot that only outputs JSON. You reply in JSON format with the field 'zip\_code'.

Example question: What is the zip code of the Empire State Building?

Example answer: {'zip\_code': 10118}

Question: What is the zip code of Menlo Park?

Response:

json

```
{'zip_code': 94025}
```



## Program-aided language models

LLMs, by nature, aren't great at performing calculations. While LLMs are bad at arithmetic, they're great for code generation. Program-Aided Language makes use of a model's code-generation skills by instructing the model to write code to solve calculation tasks.

### Example

Prompt:

```
Only return Python code, nothing else.
Generate python code to calculate the following:
((-5 + 93 * 4 - 0) * (4^4 + -7 + 0 * 5))
```



Response:

```
python

def calculate_expression():
 # Calculate the first part of the expression
 part1 = -5 + 93 * 4 - 0

 # Calculate the second part of the expression
 part2 = 4 ** 4 + -7 + 0 * 5

 # Calculate the final result
 result = part1 * part2

 return result

print(calculate_expression())
```



Executing the returned code provides the correct result.

## Reducing hallucinations

Even modern LLMs can produce hallucinations—confidently stated information that isn't supported by the source material. Meta's [Responsible Use Guide](#) is a great resource to

understand how best to prompt and address input/output risks of the language model. Refer to pages (14-17).

A well-crafted prompt can help to reduce hallucination in language models, by providing them with clear and accurate information and context.

Below are some examples of how a language model might hallucinate and some strategies for fixing the issue:

### Example 1

A language model is asked to generate a response to a question about a topic it has not been trained on. The language model may hallucinate information or make up facts that are not accurate or supported by evidence.

**Fix:** To fix this issue, you can provide the language model with more context or information about the topic to help it understand what is being asked and generate a more accurate response. You could also ask the language model to provide sources or evidence for any claims it makes to ensure that its responses are based on factual information.

### Example 2

A language model is asked to generate a response to a question that requires a specific perspective or point of view. The language model may hallucinate information or make up facts that are not consistent with the desired perspective or point of view.

**Fix:** To fix this issue, you can provide the language model with additional information about the desired perspective or point of view, such as the goals, values, or beliefs of the person or entity being addressed. This can help the language model understand the context and generate a response that is more consistent with the desired perspective or point of view.

### Example 3

A language model is asked to generate a response to a question that requires a specific tone or style. The language model may hallucinate information or make up facts that are not consistent with the desired tone or style.

**Fix:** To fix this issue, you can provide the language model with additional information about the desired tone or style, such as the audience or purpose of the communication. This can help the language model understand the context and generate a response that is more consistent with the desired tone or style.